CIS 3300 · THE SPEEDRUN LEADERBOARD

Checkpoint 2

Skipping Bad Lines

Starts from: SpeedrunCheckpoint1. **Ends at:** SpeedrunCheckpoint2. **Prepares you for:** Lab W14, the rest of Part 1.

At the end of Checkpoint 1, one bad line still stopped the whole program. By the end of this checkpoint, the program keeps every good run, skips every bad line, writes each bad line and the reason it was rejected to a file, and logs what it did. The plan has three pieces:

1. an exception class that describes one bad line;
2. a parser that turns *any* problem with a line into that exception;
3. a loop that catches the exception for each line and moves on.

This is the habit the Designing with Exceptions notes call *throw early, catch late*: the parser knows a line is bad, so it throws; the loop knows the program should keep going, so it catches.

Before you start, see the *Getting Started* guide for how to import a project into Eclipse and for the rules the leaderboard uses for each field.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

Step 1: Write the exception class

In the `speedrun` package, create a new class named `RunParseException`:

```
package speedrun;

public class RunParseException extends Exception {
    private static final long serialVersionUID = 1L;

    private final int lineNumber;
    private final String line;

    public RunParseException(int lineNumber, String line, String message) {
        super("line " + lineNumber + ": " + message);
        this.lineNumber = lineNumber;
        this.line = line;
    }

    public RunParseException(int lineNumber, String line, String message,
            Throwable cause) {
        super("line " + lineNumber + ": " + message, cause);
```

```
        this.lineNumber = lineNumber;
        this.line = line;
    }

    public int getLineNumber() {
        return lineNumber;
    }

    public String getLine() {
        return line;
    }
}
```

- **It extends `Exception`, so it is checked.** A bad line comes from the file, not from a mistake in the code. Even a correct program meets one, and the caller can always do something sensible about it: skip the line. Making it checked means the compiler forces every caller to plan for it.
- **Each constructor calls `super(...)` first,** passing the message up to `Exception`. That is ordinary constructor chaining, and it is why `getMessage()`, which this class inherits, returns text such as `line 7: ...`.
- **The second constructor takes a cause.** When another exception revealed the problem, such as a `NumberFormatException`, it is passed along so that nothing about the original problem is lost. Step 3 shows how to get it back.
- **It keeps the line itself,** not only its number, so the rejected-lines file can show the moderators exactly what was wrong.
- `serialVersionUID` is there because every exception is serializable, as the serialization notes explain. Without it, Eclipse shows a warning.

Step 2: Write the parser

In the `speedrun` package, create a class named `RunParser` with one static method that turns one line into a `Run`. Every problem with the line must come out of it as a `RunParseException`.

```
package speedrun;

import java.time.Duration;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;

public final class RunParser {

    private RunParser() { }

    public static Run parse(int lineNumber, String line) throws RunParseException {
        String[] fields = line.split(",", -1);
        if (fields.length != 6) {
            throw new RunParseException(lineNumber, line,
                    "expected 6 fields but found " + fields.length);
        }
```

```
        Duration time;
        try {
            time = RunTime.parse(fields[3]);
        } catch (NumberFormatException e) {
            throw new RunParseException(lineNumber, line,
                    "time has a part that is not a number: \"" + fields[3] + "\"", e);
        } catch (IllegalArgumentException e) {
            throw new RunParseException(lineNumber, line, e.getMessage(), e);
        }

        LocalDate date;
        try {
            date = LocalDate.parse(fields[4]);
        } catch (DateTimeParseException e) {
            throw new RunParseException(lineNumber, line,
                    "date is not a real date: \"" + fields[4] + "\"", e);
        }

        boolean verified;
        if (fields[5].equals("yes")) {
            verified = true;
        } else if (fields[5].equals("no")) {
            verified = false;
        } else {
            throw new RunParseException(lineNumber, line,
                    "verified must be yes or no, but was \"" + fields[5] + "\"");
        }

        try {
            return new Run(fields[0], fields[1], fields[2], time, date, verified);
        } catch (IllegalArgumentException e) {
            throw new RunParseException(lineNumber, line, e.getMessage(), e);
        }
    }
}
```

- **Some problems it finds itself,** such as the wrong number of fields or a verified value that is not `yes` or `no`. For those it simply throws.
- **Other problems Java finds first,** and reports with its own exceptions: `RunTime.parse` throws `NumberFormatException` or `IllegalArgumentException`, `LocalDate.parse` throws `DateTimeParseException`, and the `Run` constructor throws `IllegalArgumentException`. The parser catches each one and throws a `RunParseException` instead, passing the original as the cause. This is called *translating* an exception, and keeping the cause is called *chaining*. The message changes from Java's terms to the leaderboard's terms, and the original is still there for a programmer who needs it.
- **The order of the two `catch` blocks matters again.** `NumberFormatException` is a subclass of `IllegalArgumentException`, so it must come first, for the same reason as in Checkpoint 1. In the other order, the compiler refuses:

```
error: exception NumberFormatException has already been caught
```

- **`split(",", -1)`.** The second argument is a *limit*. Plain `split(",")` uses a limit of 0, which drops empty fields at the end of a line; any negative limit, such as `-1`, keeps them. Without it, a line that ends in a comma, such as one with an empty verified field, would be reported as having 5 fields, instead of having a verified value that is not `yes` or `no`.
- **The verified check** fixes the problem from Check yourself question 4 in Checkpoint 1. A line with `maybe` is now rejected instead of quietly becoming unverified.
- **The method declares `throws RunParseException`** but never catches it. Catching it is the loop's job.

Step 3: Try the parser

Add these lines at the very top of `main`, with `import speedrun.RunParser;` and `import speedrun.RunParseException;` at the top of the file:

```
String good = "NovaByte,Super Mario 64,16 Star,16:51.5,2025-09-03,no";
String bad = "Kestrel,Celeste,Any%,28:4x.1,2025-10-02,yes";
try {
    System.out.println(RunParser.parse(2, good));
    System.out.println(RunParser.parse(7, bad));
} catch (RunParseException e) {
    System.out.println(e.getMessage());
    System.out.println("Cause: " + e.getCause());
}
```

```
Run[player=NovaByte, game=Super Mario 64, category=16 Star, time=PT16M51.5S, date=2025-09-03,
    verified=false]
line 7: time has a part that is not a number: "28:4x.1"
Cause: java.lang.NumberFormatException: For input string: "4x.1"
```

The good line became a `Run`. The bad line became a `RunParseException` whose message speaks the leaderboard's language, and `getCause()` still returns the `NumberFormatException` that started it. Delete these lines before going on.

Step 4: Catch inside the loop

Now replace the loop in `main`. Instead of printing each run, the program keeps the good runs in one list and the exceptions for the bad lines in another, then writes the bad lines to a file. Also change the file name to `files/runs.csv`, since the program no longer crashes on it.

```
String fileName = "files/runs.csv";
try {
    List<String> lines = Files.readAllLines(Path.of(fileName));
    List<Run> runs = new ArrayList<>();
    List<RunParseException> rejected = new ArrayList<>();

    // Line 0 is the header. Element i of the list is line i + 1 of the file.
    for (int i = 1; i < lines.size(); i++) {
        try {
            runs.add(RunParser.parse(i + 1, lines.get(i)));
```

```
        } catch (RunParseException e) {
            rejected.add(e);
        }
    }

    writeRejected(fileName, rejected, "files/rejected.txt");
} catch (NoSuchFileException e) {
    // ... the two catch blocks, exactly as before ...
```

Add this method to the class, below `main`, and add imports for `java.io.PrintWriter` and `java.util.ArrayList`:

```
private static void writeRejected(String source, List<RunParseException> rejected,
        String outFile) throws IOException {
    try (PrintWriter out = new PrintWriter(outFile)) {
        out.println("Rejected lines from " + source + ": " + rejected.size());
        for (RunParseException e : rejected) {
            out.println();
            out.println(e.getMessage());
            out.println("    " + e.getLine());
        }
    }
}
```

- **The `try` block is inside the loop.** When a line is bad, the `catch` records it, and the loop goes on to the next line. If the `try` block surrounded the whole loop instead, the first bad line would end the loop, which is exactly the problem we started with.
- **The line number is `i + 1`.** Element `i` of the list is line `i + 1` of the file, so the numbers in the output match what a text editor shows.
- **`writeRejected` declares `throws IOException`** instead of catching it, because `main` already catches `IOException` and is the right place to report it.
- **`runs` is not used yet.** Later checkpoints build the leaderboard from it.

Run the program. Nothing appears in the console, because the program no longer prints the runs. In Eclipse, click the `files` folder and press F5 to refresh it, then open `rejected.txt`. It begins like this, and lists all nine bad lines:

```
Rejected lines from files/runs.csv: 9

line 7: time has a part that is not a number: "28:4x.1"
    Kestrel,Celeste,Any%,28:4x.1,2025-10-02,yes

line 11: time must be greater than 0, but was 0:00.0
    Moss,Portal,Inbounds,0:00.0,2025-11-14,yes

line 16: date 2099-05-01 is in the future
    Rook,Hollow Knight,Any%,35:12.4,2099-05-01,yes
```

The line 16 message came from the `Run` record's constructor, and the line 11 message too. The

parser translated both without changing their wording.

Step 5: Log what the program does

`rejected.txt` is a report for the leaderboard's moderators: these lines need fixing. The programmer needs something different: a record of what the program did while it ran. That is what a logger is for. Add a logger to the class, as the Logging notes show, with `import java.util.logging.Logger;` at the top:

```
public class SpeedrunReport {
    private static final Logger LOG = Logger.getLogger(SpeedrunReport.class.getName());
```

Then add four lines to `main`: the first line of `main`, a message before the file is read, one in the `catch` block in the loop, and one after the rejected lines are written.

```
public static void main(String[] args) {
    System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s%6$s%n");

    String fileName = "files/runs.csv";
    try {
        LOG.info("Loading " + fileName);
        // ... reading the lines and creating the lists, as before ...
            } catch (RunParseException e) {
                rejected.add(e);
                LOG.warning("Rejected " + e.getMessage());
            }
        }

        writeRejected(fileName, rejected, "files/rejected.txt");
        LOG.info("Loaded " + runs.size() + " runs and rejected "
                + rejected.size() + " lines");
```

The `System.setProperty` line sets the short, one-line format from the Logging notes. The notes set it as a VM argument in the run configuration instead. Setting it in code does the same thing, and it travels with the project when you import it. It must run before the first message is logged, so it goes first.

Run the program. Eclipse shows log messages in red, because they go to the standard error stream:

```
INFO: Loading files/runs.csv
WARNING: Rejected line 7: time has a part that is not a number: "28:4x.1"
WARNING: Rejected line 11: time must be greater than 0, but was 0:00.0
WARNING: Rejected line 16: date 2099-05-01 is in the future
WARNING: Rejected line 21: category is required
WARNING: Rejected line 26: expected 6 fields but found 5
WARNING: Rejected line 31: verified must be yes or no, but was "maybe"
WARNING: Rejected line 36: minutes and seconds must be below 60, but time was "1:44:61.0"
WARNING: Rejected line 40: time must be greater than 0, but was -34:08.5
WARNING: Rejected line 44: date is not a real date: "2026-02-30"
INFO: Loaded 41 runs and rejected 9 lines
```

Every bad line was rejected for its own reason, and all 41 good runs are kept, including the 36 that

the starter never reached. A rejected line is a `WARNING`, because the program worked around it; loading the file is `INFO`, a normal milestone. Those are the levels from the Logging notes' table.

Your project should now match SpeedrunCheckpoint2.

Check yourself

1. Why is `RunParseException` checked rather than unchecked?
2. What would happen if the `try` block in Step 4 surrounded the whole `for` loop instead of the body of the loop?
3. Code that catches a `RunParseException` for line 7 wants to know which Java exception started the problem. How does it find out?
4. Line 21 has a blank category. Which part of the program noticed, and how did the reason reach `rejected.txt`?
5. The Logging notes warn about *log injection*: input that contains a line break can forge a fake log entry. Could a line in `runs.csv` do that here?

Answers

1. A bad line comes from the file, not from a mistake in the code, so even a correct program meets one, and the caller can recover by skipping it. Making it checked means the compiler forces every caller to plan for it.
2. The first bad line, line 7, would end the loop. Every line after it, good or bad, would never be read, just as in the starter.
3. It calls `getCause()`, which returns the `NumberFormatException` the parser passed to the constructor.
4. The `Run` record's compact constructor threw an `IllegalArgumentException` with the message `category is required`. The parser caught it and threw a `RunParseException` with the same message and the original as the cause, and the loop caught that and added it to the list that `writeRejected` prints.
5. No. `Files.readAllLines` splits the file at its line breaks, so no line can contain one. Text from other sources, such as a name someone types, can, which is why the habit of cleaning untrusted text before logging it still matters.